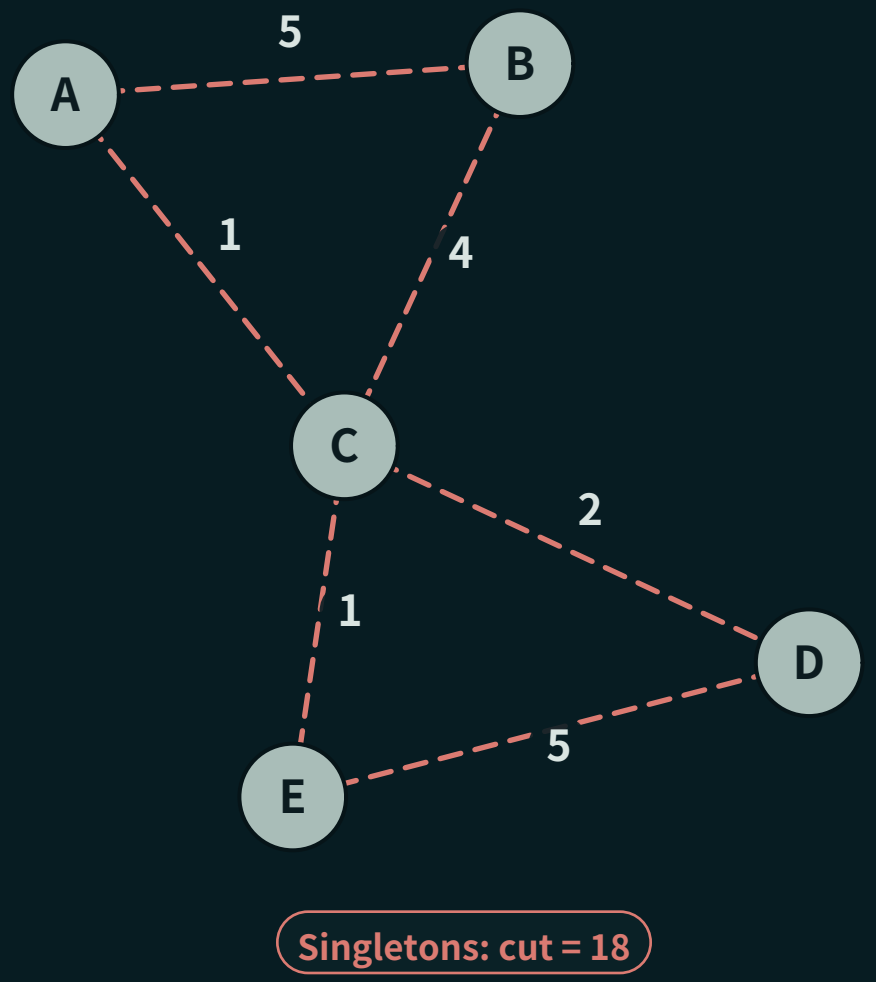
Merge, and merge again

Greedy pair merge · agglomerative clustering

Learning Clustering · original author thy031 (Issue #2)

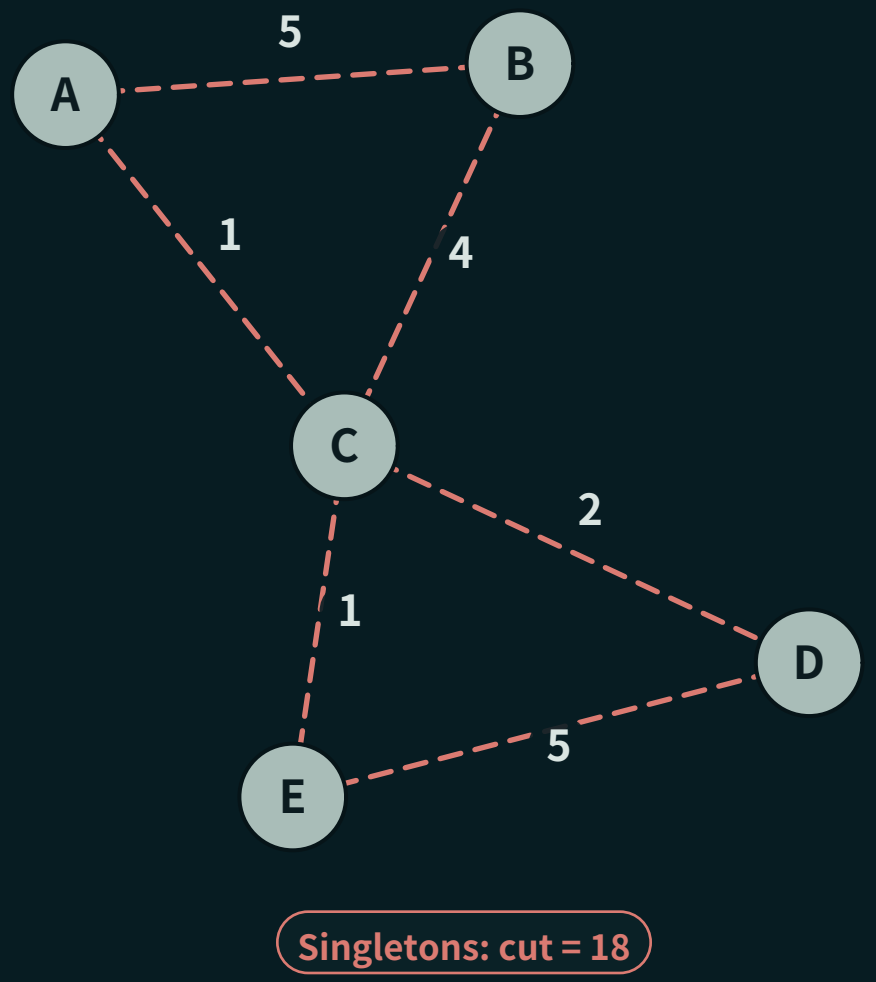
The problem

- Input: a weighted undirected graph, a target K, an optional capacity C
- Each step: merge the closest legal pair with affinity > 0
- Stop: the count reaches K, or no legal positive pair remains
- Output: clusters, cut, whether K is reached, stop reason, a record of each step
- Never force a merge to reach K, and never undo a merge



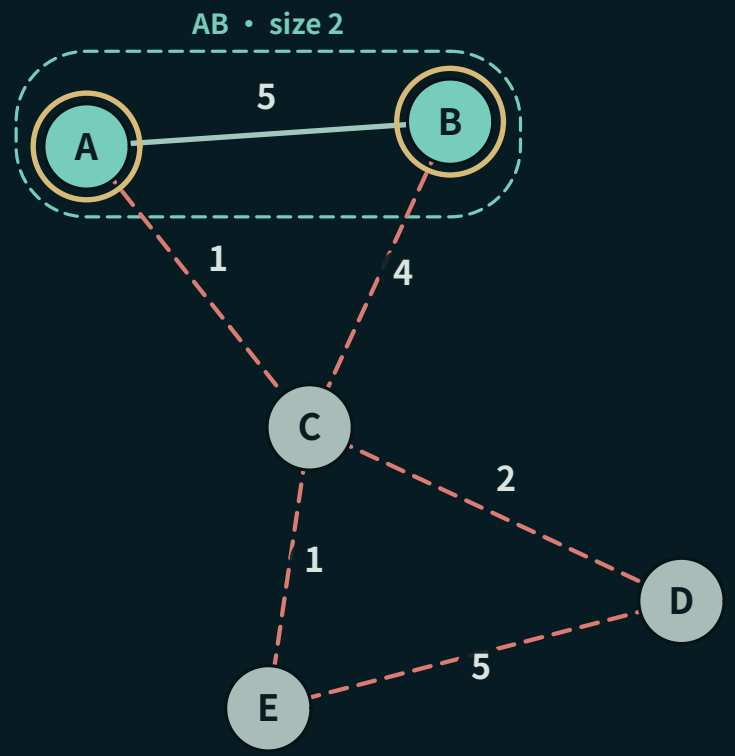
Singletons, K and the affinity between clusters

- Singletons: at the start every node is its own cluster, and cut = all 18 of the weight
- Inter-cluster affinity: the total weight of every wire between two clusters
- Contraction: wires between the two become inside wires, and parallel wires add up
- Every merge lowers the cut by exactly that merge's affinity



The selection key and the tie rule

- Selection key (−affinity, members of A, members of B): compare field by field, take the smallest
- The first field is negated: higher affinity comes first
- Step 1: A–B and D–E are both 5; compare A with D, so A–B
- cut 18 → 13

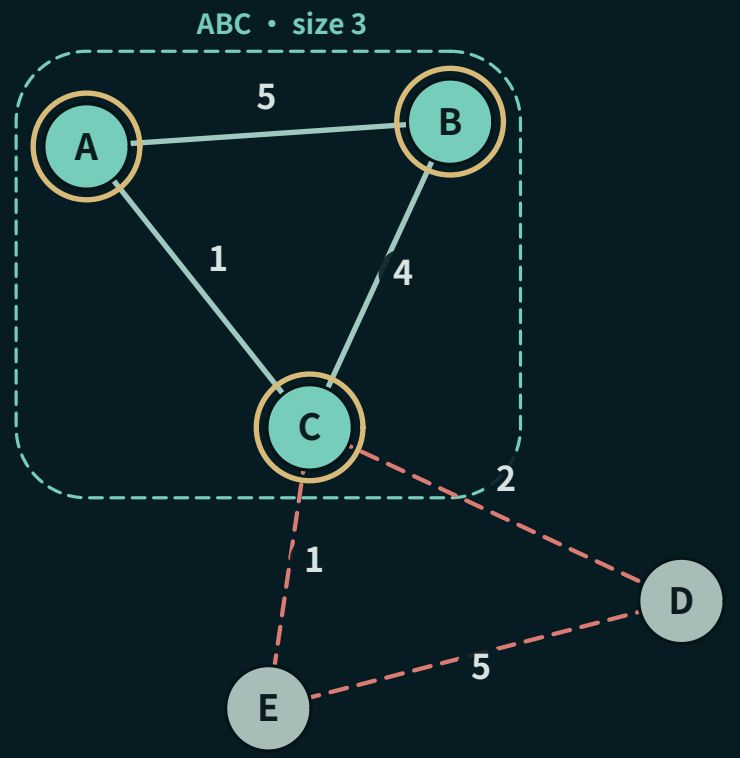


Pair	Affinity
A–B	5
D–E	5
B–C	4
C–D	2

cut 18 → 13

Contraction: parallel wires add up

- A and B contract into the supernode AB (the core illustration by the original author, thy031)
- AB–C = A–C 1 + B–C 4 = 5
- Another tie at 5 with D–E: [A, B] < [D], so AB–C
- cut 13 → 8



Pair	Affinity
AB–C	5
D–E	5
C–D	2
C–E	1

cut 13 → 8

The full path: 18 → 13 → 8 → 3

- Step 3: D–E = 5, cut 8 → 3, the count reaches 2: stop
- Result: ABC | DE
- Remaining cut: C–D 2 + C–E 1 = 3
- With K = 3 it stops after step 2: ABC | D | E, cut 8

Step	Merge	Affinity	cut
0	—	—	18
1	A + B	5	13
2	AB + C	5	8
3	D + E	5	3

The original page's comparator bug (fixed)

- Original JS: if (!best || cand < best), where cand is an array
- JavaScript turns arrays into strings before comparing
- "-1,A,C" < "-5,A,B": affinity 1 counts as better
- Four settings give 9/16/12/16; correct is 3/8/8/3
- Fix: compare field by field; regression tests 10 vs 2, 2.5 vs 2

[−5, "A", "B"] → "-5,A,B"

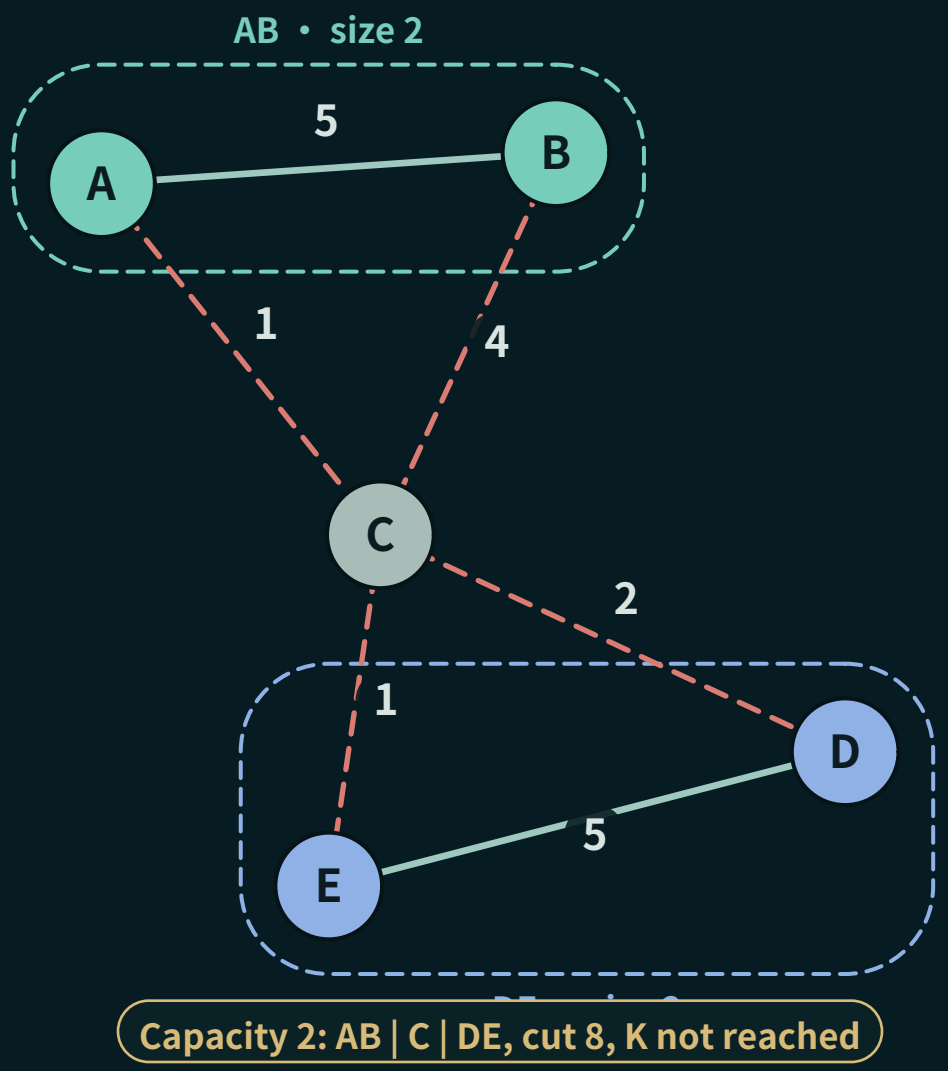
[−1, "A", "C"] → "-1,A,C"

Character 1 < 5: A–C, affinity 1, looks better

Setting	Original	Correct
K=2	9	3
K=3	16	8
K=2 cap 2	12	8
K=2 cap 3	16	3

A capacity preview and the stopping state

- Capacity C = 2: AB + C has size 3 > 2, blocked
- D–E merges instead; after that AB+C and C+DE are both over capacity
- Stop: AB | C | DE, 3 clusters, cut 8
- Legal (every cluster ⩽ 2) but K = 2 not reached
- Capacity 3: the same path as with no capacity



The pseudocode, line by line

- Line 2: stop once the count reaches K
- Lines 3–4: list positive candidates, drop those over capacity
- Line 5: with no legal candidate, stop and say so
- Line 6: compare selection keys field by field
- Line 7: contract; the cut drops by that affinity

```
1  clusters ← one singleton per node
2  while #clusters > K:
3    cands ← pairs with affinity > 0
4    legal ← cands with merged size ⩽ C
5    if legal is empty: stop (say so)
6    pick the pair with the least (−aff, P, Q)
7    contract P ∪ Q: cut drops by its affinity
8  return clusters, cut, K reached, stop reason
```

Common mistakes

- Comparing candidate arrays as strings: step 1 wrongly picks A–C
- Forcing an over-capacity merge to reach K
- Forgetting to add parallel wires after a contraction: AB–C taken as 4
- Seeing cut 3 and assuming greedy is always optimal

Limits and practical meaning

- No backtracking: a wrong early merge cannot be corrected (counterexample in Module 04)
- Depends on the score and the tie rule
- A naive version checks every cluster pair each step; real tools use a priority queue and local updates
- In chip design it often coarsens many cells into fewer clusters, with a limit on cluster area

Summary

Each step merges the closest legal pair; value first, names break ties.

The cut drops by each merge's affinity; stop at K or when no legal pair remains, and say so.

Next module: capacity takes centre stage. Does it not fit, or did greedy take a wrong turn?